Machine Learning Operations (MLOps) AIMLCZG523 — Assignment 01

Heart Disease Risk Prediction – End-to-End MLOps Pipeline

Name: Priya S Kumar

Student ID: 2025AE05763

Date: 11 October 2026

GitHub: https://github.com/hgpriya/bits_mlops_assignment

Demo video: MLOPS_Assignment1.mp4

Contents

1 Introduction 3
2 Setup and Installation 3
3 Data Acquisition and Exploratory Data Analysis 3
3.1 Acquisition and cleaning 3
3.2 Exploratory findings 4
4 Feature Engineering and Model Development 5
4.1 Preprocessing pipeline 5
4.2 Training and hyperparameter tuning 5
4.3 Model selection 5
5 Experiment Tracking with MLflow 6
6 Model Packaging and Reproducibility 7
7 CI/CD Pipeline and Automated Testing 7
7.1 Unit tests 7
7.2 GitHub Actions workflow 7
8 API and Containerisation 8
9 Kubernetes Deployment 9
10 Monitoring and Logging 9
11 Architecture 10
12 Conclusion and Limitations 11

1 Introduction

This report documents an end-to-end MLOps pipeline for predicting the presence of heart disease. A binary classifier is trained on the UCI Heart Disease dataset (Cleveland subset: 303 patients, 13 clinical features) and delivered as a tested, experiment-tracked, containerised REST API that is deployed on Kubernetes and monitored with Prometheus and Grafana. Every stage, from data acquisition and EDA through modelling, tracking, packaging, CI/CD, deployment and monitoring, is scripted and reproducible from the public GitHub repository.

2 Setup and Installation

Development took place in GitHub Codespaces, which provides a consistent container-based environment with Docker pre-installed, and a Python virtual environment isolates the project packages. Dependencies are split across four files (Table 1) so that training, development and the serving image each install only what they need.

Table 1. Dependency files

File	Contents and purpose
requirements.txt	Runtime, training and serving libraries
requirements-dev.txt	requirements.txt plus pytest, pytest-cov, httpx and flake8
requirements-api.txt	Slim dependency set for the serving image
environment.yml	Equivalent Conda environment

The complete workflow runs with a handful of commands:

```
python -m venv .venv && source .venv/bin/activate

pip install -r requirements-dev.txt

python data/download_data.py      # fetch UCI Cleveland data

python scripts/run_eda.py         # cleaning report and EDA figures

python scripts/train.py           # tuning, MLflow logging, model export

pytest -v

docker build -t heart-risk-api:1.0.0 .
```

The Makefile wraps the same steps as make eda, make train, make test, make lint, make docker-build, make k8s-deploy and make monitor-up, so local runs, CI and reviewers use identical commands.

3 Data Acquisition and Exploratory Data Analysis

3.1 Acquisition and cleaning

data/download_data.py fetches the Cleveland file directly from the UCI repository. The cleaning steps implemented in src/heart_mlops/data.py are summarised in Table 2.

Table 2. Data cleaning steps

Issue	Finding	Treatment
Missing values in ca	'?' in 4 rows (1.32%)	Most-frequent imputation
Missing values in thal	'?' in 2 rows (0.66%)	Most-frequent imputation
Target num (0–4)	Graded diagnosis	Binarised: num > 0 → 1 (disease)

Duplicate rows	0 found	Dropped (none removed)
Clinical range checks	No values out of range	No changes required

3.2 Exploratory findings

The classes are reasonably balanced: 54.1% of patients have no disease and 45.9% have disease, so all splits are stratified and no resampling is applied. Mean age is 54.4 years (range 29–77) and 68% of patients are male. Figure 1 shows the class balance and the disease rate within each category of the categorical features.

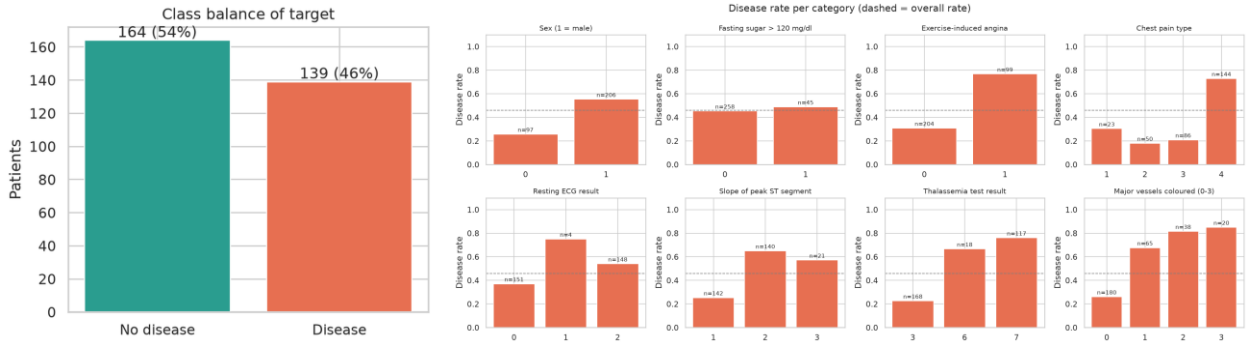


Figure 1 (left) Class balance of the binary target. (right) Disease rate by category for each categorical feature; the dashed line is the overall rate.

Table 3. Pearson correlation of each feature with the target

Feature	r	Feature	r
thal	0.522	cp	0.414
ca	0.460	slope	0.339
exang	0.432	sex	0.277
oldpeak	0.425	age	0.223
thalach	−0.417	chol / fbs	0.085 / 0.025

The thallium stress-test result(thal), the number of vessels colored(ca), exercise-induced angina(exang) and ST depression(oldpeak) are the strongest signals, while a lower maximum heart rate goes with higher risk, which motivated the engineered hr_pct_of_max feature. Cholesterol and fasting blood sugar are only weakly related to the target, and because cp, thal and slope are categorical codes their coefficients are indicative only; the per-category rates in Figure 1 are the more faithful view.

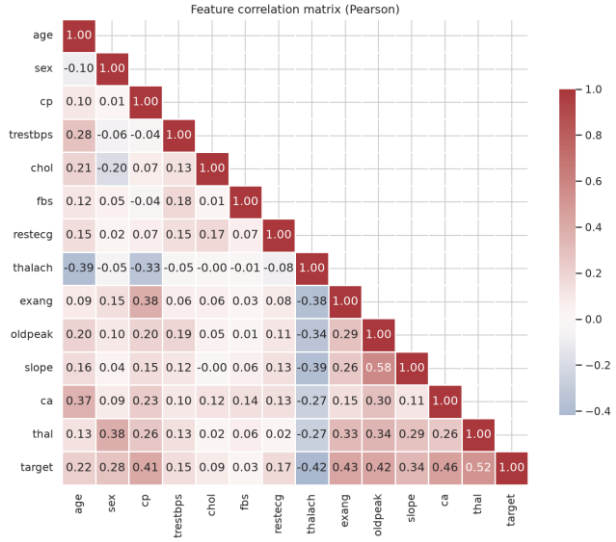


Figure 2 Correlation heatmap of the clinical features and the target.

4 Feature Engineering and Model Development

4.1 Preprocessing pipeline

All preprocessing lives inside a single scikit-learn Pipeline. A custom ClinicalFeatures transformer adds two features: hr_pct_of_max = thalach / (220 − age), the share of the age-predicted maximum heart rate reached during exercise, and bp_stage2 = trestbps ≥ 140, a stage-2 hypertension flag. A ColumnTransformer then applies median imputation and StandardScaler to numeric features, mode imputation to binary features, and mode imputation plus one-hot encoding (handle_unknown = 'ignore') to the categorical features cp, restecg, slope and thal. Because the imputers are fitted inside the pipeline, their statistics come only from the training folds and nothing leaks from validation or test data.

4.2 Training and hyperparameter tuning

The data is split 80/20 with stratification, leaving 61 test patients. Each model family is tuned with GridSearchCV using stratified 5-fold cross-validation and ROC-AUC as the objective: Logistic Regression (12 combinations of C and class_weight), Random Forest (36 combinations of n_estimators, max_depth, min_samples_leaf and max_features) and Gradient Boosting (16 combinations of n_estimators, learning_rate, max_depth and subsample), 64 trials in total. **The final model is chosen on mean CV ROC-AUC and refitted on all 303 rows**. Tables 4 to 6 report the results.

Table 4. Cross-validation results (mean ± std over 5 stratified folds)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.847 ± 0.022	0.879 ± 0.071	0.783 ± 0.062	0.824 ± 0.025	0.907 ± 0.019
Random Forest	0.814 ± 0.023	0.833 ± 0.030	0.747 ± 0.086	0.784 ± 0.040	0.897 ± 0.024
Gradient Boosting	0.826 ± 0.039	0.831 ± 0.075	0.792 ± 0.075	0.806 ± 0.043	0.876 ± 0.026

Table 5. Hold-out test results (61 patients)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.885	0.839	0.929	0.881	0.959
Random Forest	0.885	0.839	0.929	0.881	0.950
Gradient Boosting	0.902	0.867	0.929	0.897	0.955

Table 6. Best hyperparameters from the grid search

Model	Best parameters
Logistic Regression	C = 0.3, class_weight = None
Random Forest	n_estimators = 200, max_depth = None, max_features = sqrt, min_samples_leaf = 3
Gradient Boosting	n_estimators = 100, learning_rate = 0.05, max_depth = 2, subsample = 0.8

4.3 Model selection

Logistic Regression was selected because it has the highest mean CV ROC-AUC (0.907) and the lowest variance across folds. On the hold-out set all three models detect 26 of the 28 diseased patients (recall 0.929), which matters clinically because a missed case is far more costly than a false alarm.

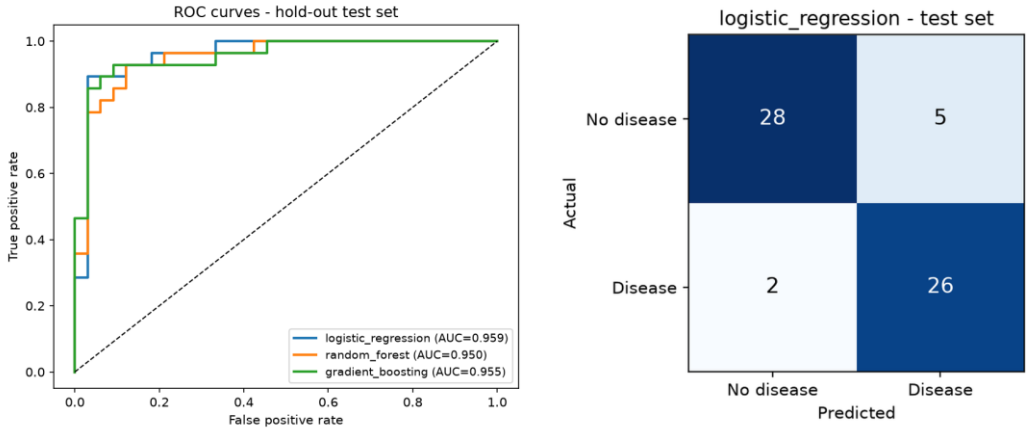


Figure 3 (left) ROC curves of the three tuned models on the hold-out test set. (right) Confusion matrix of the selected Logistic Regression model.

5 Experiment Tracking with MLflow

Experiments are tracked with MLflow 3.17.0 using a local SQLite backend under the experiment heart-disease-risk. Each training session creates a parent run. Beneath it, one nested run per model family stores the best parameters, CV and test metrics, the confusion matrix, a feature-importance plot and the full grid results as CSV, and one nested run per grid trial (64 in total) records every hyperparameter combination. The parent run logs the ROC and CV comparison plots, model_comparison.md, model_card.json and **the final MLflow model, which is registered as heart-disease-classifier**.

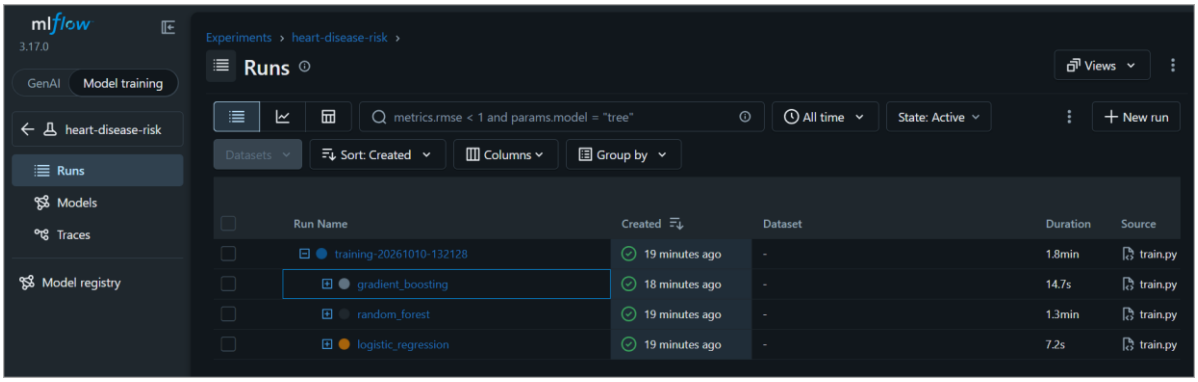


Figure 4 MLflow runs list: the parent training run with the three nested model-family runs.

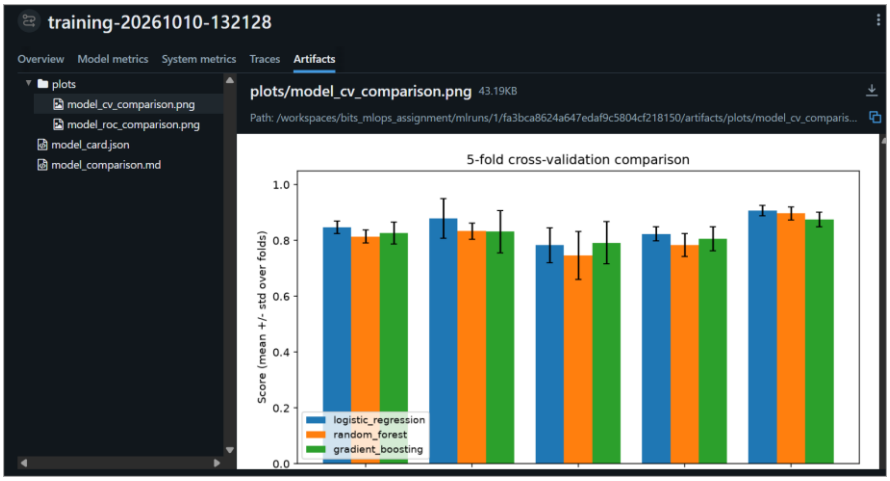


Figure 6 Artifacts of the parent run, including the CV comparison chart.

6 Model Packaging and Reproducibility

Training writes the complete fitted pipeline (preprocessing plus classifier) to models/model.joblib, so the API loads one object and cannot apply preprocessing inconsistently. Alongside it, models/model_card.json records what was trained, how it was selected and on which data (excerpt):


```
{"model_name": "logistic_regression", "selection_metric": "cv_roc_auc_mean",
 "best_params": {"C": 0.3, "class_weight": "None"},
 "metrics": {"cv_roc_auc_mean": 0.9072, "test_roc_auc": 0.9589, ...},
 "decision_threshold": 0.5, "training_rows": 303, "data_sha256": "d8f27056...",
 "python": "3.14.2", "sklearn": "1.8.0"}
```


Reproducibility rests on a fixed random seed (42) for splitting, cross-validation and the models, the data hash in the card, and version pinning: the Docker build reads the scikit-learn version from the model card and installs exactly 1.8.0. A command-line check needs no server: python scripts/predict.py --input sample_request.json returns prediction 1 with probability 0.8863 in the high risk band (Figure 7).

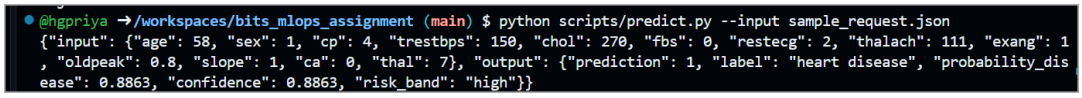


Figure 7 Command-line prediction for sample_request.json.

7 CI/CD Pipeline and Automated Testing

7.1 Unit tests

The suite contains 32 pytest unit tests covering data cleaning, feature engineering, training helpers, inference and the API, including HTTP 422 validation errors and the /metrics endpoint. All 32 pass locally (Figure 8) and in CI. Overall line coverage is 68%: api 95%, predict 97%, features 97% and data 94%. The train (31%) and plots (18%) modules score lower because that orchestration code is exercised end to end by the CI training job rather than by unit tests. flake8 enforces code style.

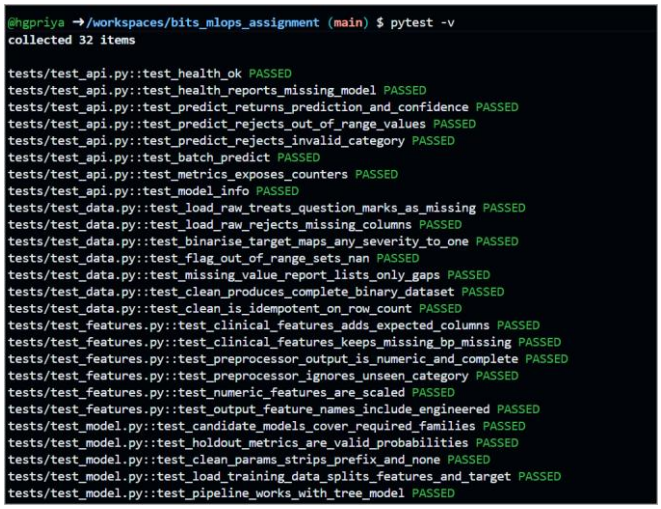


Figure 8 Local pytest run: 32 tests passed.

7.2 GitHub Actions workflow

.github/workflows/ci.yml runs on every push, pull request and manual dispatch. Four jobs run in sequence: **lint** (flake8); **test** (pytest, with JUnit XML and coverage reports uploaded); **train** (full training run, uploading the model, reports and MLflow database and writing the model comparison table into the

run summary); and **Docker build and smoke test**, which builds the image, starts the container, checks /health, /predict, a 422 validation response and /metrics, and uploads the container logs. Any failure stops the pipeline, and the training step runs with pipefail so that an error cannot be hidden by piped output. The latest run succeeded in 3 min 20 s (Figure 9).

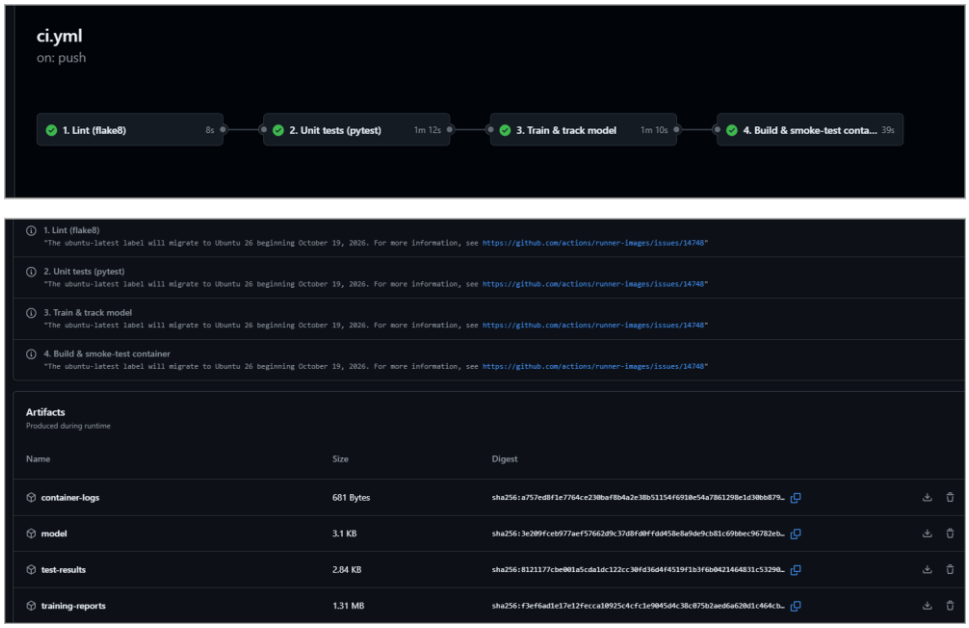


Figure 9 (top) GitHub Actions job graph: Lint 8 s → Unit tests 1 min 12 s → Train 1 min 10 s → Docker build & smoke test 39 s. (bottom) Artifacts uploaded by the run.

8 API and Containerisation

The model is served by a FastAPI application with the endpoints in Table 7. Request bodies are validated by Pydantic models that enforce clinical ranges and the allowed codes for categorical fields, so malformed input is rejected with HTTP 422 before it reaches the model. Probabilities map to risk bands: low below 0.30, moderate from 0.30 to 0.60 and high at 0.60 or above. Every response carries an x-request-id header that is also written to the logs.

Table 7. API endpoints

Endpoint	Purpose
GET /health	Liveness and readiness check
GET /model-info	Model name, parameters and metrics from the model card
POST /predict	Prediction for one patient
POST /predict/batch	Predictions for up to 100 patients
GET /metrics	Prometheus metrics

Example request to POST /predict and the response returned:

```
{"age": 58, "sex": 1, "cp": 4, "trestbps": 150, "chol": 270, "fbs": 0, "restecg": 2,
 "thalach": 111, "exang": 1, "oldpeak": 0.8, "slope": 1, "ca": 0, "thal": 7}
```

```
{"prediction": 1, "label": "heart disease", "probability_disease": 0.8863,
 "confidence": 0.8863, "risk_band": "high", "model_name": "logistic_regression"}
```

The Dockerfile builds on python:3.12-slim, installs the pinned serving dependencies, copies the code and model, runs as a non-root user, declares a HEALTHCHECK against /health and starts the app with uvicorn on port 8000.

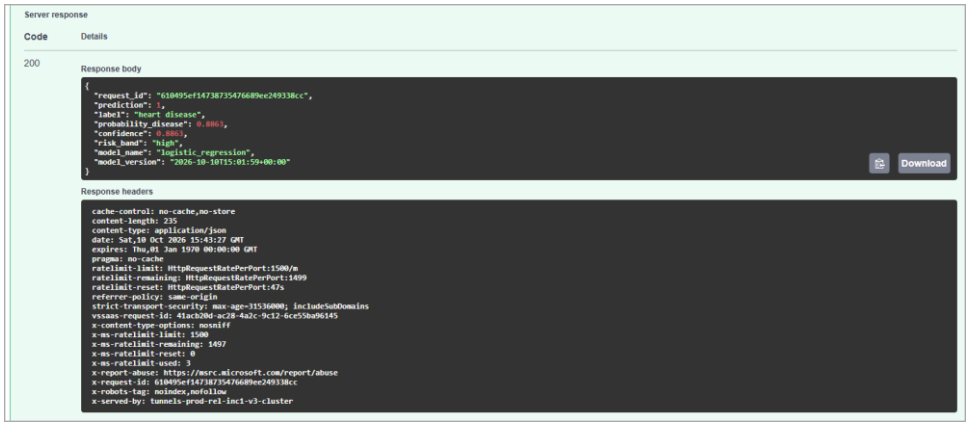


Figure 11 Swagger UI: POST /predict returning HTTP 200.

9 Kubernetes Deployment

The manifests in k8s/ define the heart-risk namespace; a Deployment with 2 replicas, a rolling-update strategy, readiness and liveness probes on /health, CPU and memory requests and limits, a non-root security context and Prometheus scrape annotations; a Service of type LoadBalancer mapping port 80 to container port 8000; an optional NGINX Ingress for heart-risk.local; and a kustomization that applies them together.

```
kind create cluster --name heart-risk

kind load docker-image heart-risk-api:1.0.0 --name heart-risk

kubectl apply -k k8s/

kubectl -n heart-risk get pods,svc

kubectl -n heart-risk port-forward svc/heart-risk-api 8080:80
```

Both pods reached the Running state. The API is reached through the port-forward above, and the smoke test passed against it (Figure 12).

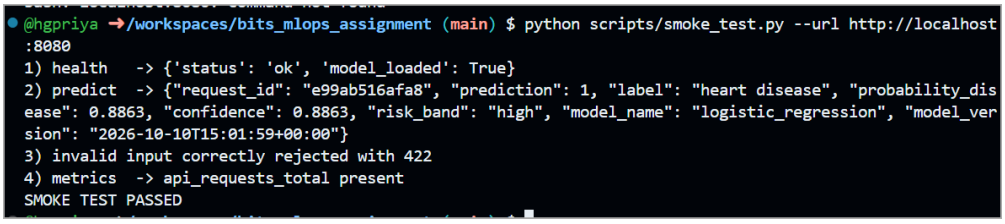


Figure 12 Smoke test against the Kubernetes deployment through kubectl port-forward.

10 Monitoring and Logging

The API writes one JSON log line per request (timestamp, level, request_id, method, path, status, latency_ms and client) plus a separate prediction log line, so logs can be filtered by request ID and parsed by any log collector; they are read with kubectl logs or docker logs. Example:

```
{"ts": "2026-10-10T16:51:07.470+00:00", "level": "INFO", "logger": "heart_api", "msg":
"request",

 "request_id": "32f0f293dbc9", "method": "POST", "path": "/predict", "status": 200,
"latency_ms": 10.19}
```

The /metrics endpoint exposes five Prometheus metrics: **api_requests_total, api_request_latency_seconds, model_predictions_total, model_probability_disease and model_loaded**. A docker compose stack runs the API, Prometheus (15 s scrape interval, 10 s timeout) and Grafana with a provisioned Heart Risk API dashboard. scripts/send_requests.py then replayed real patient rows with about 5% deliberately invalid requests. Prometheus recorded 848 successful and 52 rejected /predict requests (Figure 13), and over a 15-minute window Grafana showed 761 predictions, a peak of 4.69 requests/s, a 3.85% error rate and a /predict p95 latency of about 24 ms (Figure 14).

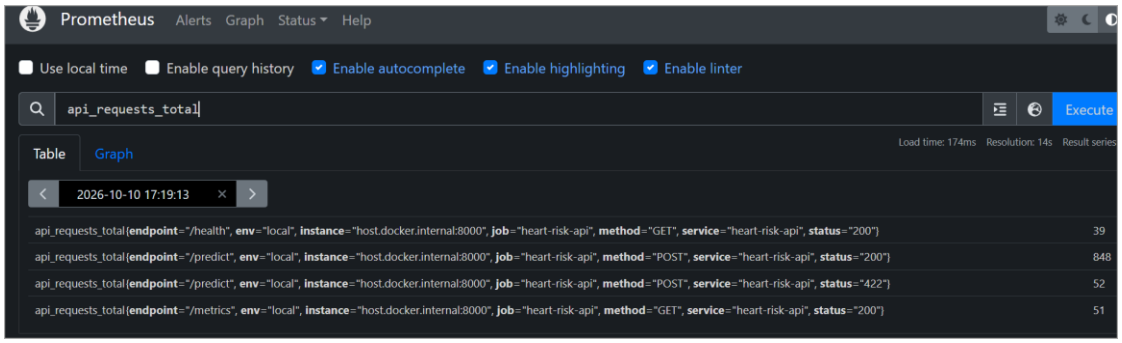


Figure 13 Prometheus query of api_requests_total by endpoint and status.



Figure 14 Grafana "Heart Risk API" dashboard (15-minute window).

11 Architecture

Figure 15 summarises the architecture. Each layer feeds the one below it; monitoring and logging run alongside the serving layer.

Layer	Components and flow
Data	UCI Heart Disease (Cleveland) → download_data.py → cleaning and EDA
▼ Training	train.py: Pipeline + GridSearchCV → MLflow tracking and model registry
▼ Artifact	models/model.joblib + model_card.json
▼ CI/CD	GitHub Actions: lint → test → train → Docker build & smoke test
▼ Container	Docker image heart-risk-api:1.0.0 (FastAPI + uvicorn)
▼ Serving	Kubernetes (kind): Deployment ×2 → LoadBalancer Service / Ingress → users
▶ Monitoring	/metrics → Prometheus → Grafana dashboard
▶ Logging	JSON request logs → stdout → kubectl logs / docker logs

Figure 15 End-to-end architecture of the heart-disease risk pipeline.

12 Conclusion and Limitations

The project delivers a reproducible path from a public clinical dataset to a monitored prediction service. The **selected Logistic Regression model reaches a CV ROC-AUC of 0.907 and a test ROC-AUC of 0.959 with a recall of 0.929**, **and every change is linted, tested, retrained and smoke-tested in CI before the image is deployed to Kubernetes and observed through Prometheus, Grafana and structured logs**. The main limitations are the small single-site dataset (303 patients), the absence of clinical validation and the use of a local kind cluster rather than a managed cloud service. Future work would add data and prediction drift monitoring, registry-based model promotion, deployment to AKS, EKS or GKE, and alerting on error rate and latency.